

CCA GAME DEVELOPMENT

# Space Shooter OOP

Mini Project 1

Module 1 — Session 4

Object-Oriented Programming & Game Architecture

---

Teacher: Annisa Hendrawan

Senior High School | Global Prestasi School

# From Procedural Code to OOP

## BEFORE

### PLAYER

- player\_x
- player\_y
- player\_speed

### LASERS

- lasers
- laser\_speed

### ASTEROIDS

- asteroids
- asteroid\_speed

### GAME LOOP

- move player
- move lasers
- spawn asteroids
- move asteroids
- collision
- draw everything

## AFTER

### Player Class

- image
- rect
- speed
- └─ update()

### Laser Class

- image
- rect
- speed
- └─ update()

### Asteroid Class

- image
- rect
- speed
- └─ update()

### Sprite Groups

- └─ all\_sprites
- └─ lasers
- └─ asteroids

### GAME LOOP

- Events
- ↓
- Create Objects
- ↓
- Update
- ↓
- Collision
- ↓
- Draw

# Today's Mission

Today, we will **refactor our existing Space Shooter** into a more organized OOP game.

At the end:

We will have a more organized OOP version of our Space Shooter.

## Our 6 Big Ideas

### 01 — Refactoring

Organize existing code without changing its main behavior.

### 02 — Class

Create a blueprint for our game objects.

### 03 — Object

Create actual game objects from our classes.

### 04 — `__init__()` & `self`

Set up and manage an object's own data.

### 05 — Sprite Groups

Manage many game objects together.

### 06 — Delta Time

Make movement depend on elapsed time.

# Mini Project 1: Space Shooter OOP

We already have a working Space Shooter.

Today, we will **change its code structure**, not its main game idea.

Our game still needs to:

- Move the Player
- Shoot Lasers
- Spawn Asteroids
- Move objects
- Detect collisions
- Keep running in a Game Loop



# Look at the Existing Code

A simple Space Shooter may store game data like this:

```
main.p
y

player_x = 375
player_y = 500
player_speed = 300

lasers = []
laser_speed = 500

asteroids = []
asteroid_speed = 200
```

And the main game loop controls:

- Player movement
- Laser movement
- Asteroid movement
- Asteroid spawning
- Collision
- Drawing

## Question

**What happens when our game gets bigger?**

# Big Idea #1: Refactoring

---

## What Is Refactoring?

**Refactoring** means reorganizing existing code without changing its intended behavior.

Our goal:

- Organize the code
- Make responsibilities clearer
- Make the game easier to expand

We are changing the structure,  
not the main game idea.

# From Procedural Code to OOP

## BEFORE

```
player_x = 375
player_y = 500
player_speed = 300

lasers = []
asteroids = []

# Main loop controls everything
```

## AFTER

```
class Player:
    ...

class Laser:
    ...

class Asteroid:
    ...
```

## What changed?

**Before:** Different game objects are mixed together.

**After:** Each game object has its own structure.

Main idea

**Each part of the game gets a clear responsibility.**

# Organizing a Game Object

A **class** is a blueprint.

An **object** is the actual thing created from that blueprint.

```
class Player:

    def __init__(self):
        self.speed = 300
```

Create the object:

```
player = Player()
```

## Remember

`class Player`

→ Blueprint

`player = Player()`

→ Object

`__init__()`

→ Initial setup (**initialize**)

`self`

→ The current object

# Refactoring the Player

## BEFORE

```
player_x = 375
player_y = 500
player_speed = 300

keys = pygame.key.get_pressed()

if keys[pygame.K_LEFT]:
    player_x -= player_speed * dt
if keys[pygame.K_RIGHT]:
    player_x += player_speed * dt
```

## AFTER

```
class Player:
    def __init__(self):
        self.x = 375
        self.y = 500
        self.speed = 300

    def update(self, dt):
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            self.x -= self.speed * dt
        if keys[pygame.K_RIGHT]:
            self.x += self.speed * dt
```

## What changed?

**Before:** Player data and behavior are in the main program.

**After:** Player data and behavior are inside the Player class.

player\_x → self.x   player\_y → self.y   player\_speed → self.speed

# Turning the Player into a Sprite

A Pygame Sprite is a game object that Pygame can manage.

```
class Player(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()

        self.image = pygame.Surface((50, 50))
        self.image.fill((50, 200, 255))

        self.rect = self.image.get_rect(
            center=(400, 525)
        )
        self.speed = 300
```



Key parts: `pygame.sprite.Sprite`  
Pygame's base Sprite class

`super().__init__()`

# From Manual Drawing to Sprite

## BEFORE

```
player_x = 375
player_y = 500

pygame.draw.rect(
    screen,
    (50, 200, 255),
    (player_x, player_y, 50, 50)
)
```

## AFTER

```
self.image = pygame.Surface((50, 50))
self.image.fill((50, 200, 255))

self.rect = self.image.get_rect(
    center=(400, 525)
)
```

## What changed?

**Before:** We manually draw the Player using coordinates.

**After:** The Sprite stores its image (Appearance) and rect (Position).

Now Pygame can manage the Player as a Sprite.

# Your Turn: Refactor the Player

Open your existing Space Shooter. Move the Player-related code into the class.

## 1. The Structure

```
class Player(pygame.sprite.Sprite):  
    # Put logic here  
    ...
```

## 2. Include Attributes

- Player starting position
- Player speed
- Player movement
- image
- rect

**Then test: Does the Player still move correctly?**

# Big Idea #5: Sprite Groups

## Managing Many Game Objects

A **Sprite Group** is a container that manages multiple Sprites.



```
all_sprites = pygame.sprite.Group()
lasers = pygame.sprite.Group()
asteroids = pygame.sprite.Group()
```

Add objects:

```
all_sprites.add(player)
all_sprites.add(laser)
all_sprites.add(asteroid)
```

Then manage them together

```
all_sprites.update(dt)
all_sprites.draw(screen)
```

Main idea

**Group = manage many Sprites together**

# Managing Many Objects

## BEFORE

```
lasers = []
asteroids = []

for laser in lasers:
    laser[1] -= laser_speed * dt

for asteroid in asteroids:
    asteroid[1] += asteroid_speed * dt
```

## AFTER

```
all_sprites = pygame.sprite.Group()

all_sprites.add(player)
all_sprites.add(laser)
all_sprites.add(asteroid)

all_sprites.update(dt)
all_sprites.draw(screen)
```

## What changed?

**Before:** We manually manage different lists and loops.

**After:** The Sprite Group helps manage our Sprites together.

# Big Idea #6: Delta Time

## Making Movement Time-Based

dt = **Delta Time**.

**Delta** means **difference or change**.

```
dt = clock.tick(60) / 1000
```

dt tells us how much time passed between frames.

For movement:

**movement = speed \* dt**

BEFORE

```
self.rect.y += self.speed
```

AFTER

```
self.rect.y += self.speed * dt
```

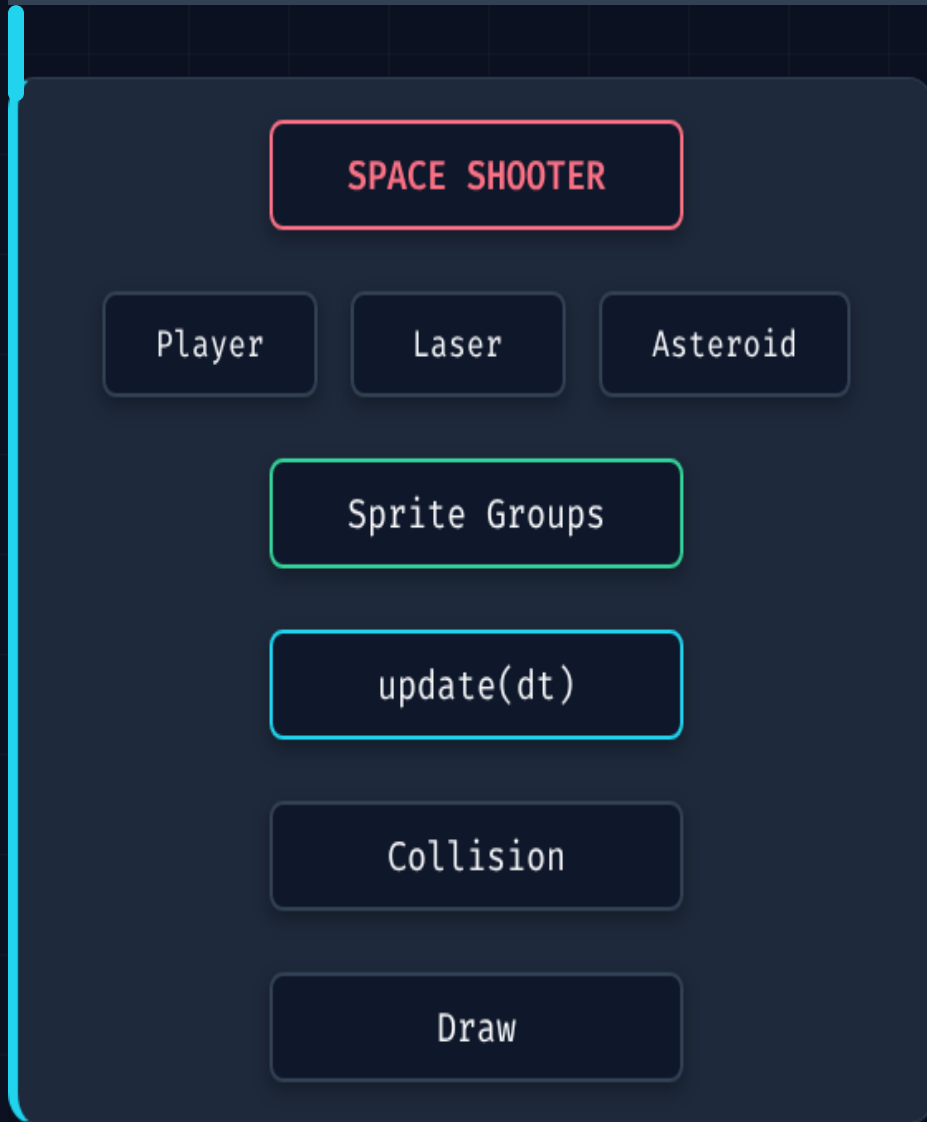
What changed?

**Before:** Movement depends on frames.

**After:** Movement depends on elapsed time.

**Speed × Time = Movement**

# Space Shooter OOP Architecture



## The Main Game Loop

The main game loop now focuses on high-level logic:

1. Events



2. Create Objects



3. Update



4. Collision



5. Draw

# Refactor Your Space Shooter

Now apply the ideas to **your own existing game**.

## Required:

- Player class
- Laser class
- Asteroid class
- Pygame Sprites
- Sprite Groups
- `update(dt)`
- Delta Time
- Working collision

Important

**Your game should still work after refactoring.**



# Customize Your Space Shooter

Once your OOP structure works, make the game your own.



## Change

- Player design
- Laser design
- Asteroid design
- Movement speed
- Spawn rate
- Background



## + Add

- Score
- Lives
- Power-ups
- New enemies
- Sound effects
- New weapons

# Final Recap: The 6 Big Ideas

---

## 01 — Refactoring

Reorganize existing code cleanly.

## 02 — Class

A blueprint for game objects.

## 03 — Object

An actual thing created from a class.

## 04 — `__init__()` & `self`

Set up and manage an object's own data.

## 05 — Sprite Groups

Manage many Sprites together.

## 06 — Delta Time

Make movement depend on elapsed time.

THINK LIKE A GAME DEVELOPER

When your game becomes bigger, ask:

# "Who should be responsible for this?"

Player movement → **Player**

Manage many Sprites → **Sprite Group**

Laser movement → **Laser**

Asteroid movement → **Asteroid**

Time between frames → **dt**

Good game architecture means:

## Each part of the game has a clear responsibility.